

Document number: P2521R0
Date: 2022-01-17
Audience: SG21
Reply-to: Gašper Ažman <gasper dot azman at gmail dot com>
Joshua Berne <jberne4 at bloomberg dot net>
Bronek Kozicki <brk at spamcop dot net>
Andrzej Krzemiński <akrzemi1 at gmail dot com>
Ryan McDougall <mcdougall dot ryan at gmail dot com>
Caleb Sunstrum <caleb dot sunstrum at gmail dot com>

Contract support — Working Paper

This paper is a yet another proposal to add a minimum contract support framework to C++. It proposes nothing that hasn't already been described in either [\[P2388R4\]](#) or [\[P2461R1\]](#).

The goal in this paper is to structure the proposal in a different way in order to reflect what SG21 has consensus on and what remains a controversy. We treat the following as open issues:

1. The choice of syntax.
2. Side effect elision and duplication.
3. The treatment of non-reference function parameters referenced in postconditions.

We assume that the reader is already familiar with [\[P2388R4\]](#).

1. Motivation

{mot}

The motivation for adding contract support framework to C++ is to enable the programmers to define in a formal way what constitutes a contract violation (and therefore a bug) in their programs. This information can be later used by different tools to perform static or dynamic analysis of the program, add instrumentation code, or generate documentation or programmer hints in the IDE. It has been described in more detail in [\[P2388R4\]](#).

The motivation for producing another paper is to focus on documenting the consensus of SG21.

2. Notation

{not}

Because the choice of syntax for contract annotations has no consensus yet, in this paper we use placeholder notation:

```
int select(int i, int j)
    PRE(i >= 0)                // precondition
    PRE(j >= 0)
    POST(r: r >= 0)            // postcondition; r names the return value
{
    ASSERT(_state >= 0);       // assertion; not necessarily an expression

    if (_state == 0) return i;
    else return j;
}
```

3. Proposed characteristics

{pro}

3.1. Three declarations: preconditions, postconditions and assertions

{pro.tri}

We propose that all three types of declarations are included in the minimum contract support:

1. Preconditions.
2. Postconditions.
3. Assertions.

Although it is possible to add only preconditions to the language and gain minimal benefit, we believe that only the three components added together bring sufficient value to warrant the modification of the language. We also believe that the syntax and semantics of preconditions must be compatible with these of the postconditions. So even if the preconditions were to be added in isolation, we would have to have a polished design for postconditions. This means that preconditions are blocked on the postcondition design even for the "only preconditions" variant.

3.2. Two translation modes

{pro.mod}

We propose that there are two modes that a translation unit can be translated in:

1. **No_eval**: compiler checks the validity of expressions in contract annotations, but the annotations have no effect on the generated binary. Functions appearing in the predicate are odr-used.
2. **Eval_and_abort**: each contract annotation is checked at runtime. The check evaluates the corresponding predicate; if the result equals `false`, the program is stopped an error return value.

The implementation may, but does not have to, allow the translation of different translation units in different modes. Too many modes are not necessary for the minimum contract implementation. The *No_eval* is required to provide no-overhead guarantee. The *Eval_and_abort* is required to actually assign any semantics to contract annotations.

3.3. Name look-up

{pro.lok}

We propose that names referred in preconditions and postconditions are looked up as if they appeared in a `noexcept` specification, if the function had one. In particular, this means that private members can appear in pre/post-conditions.

Programming guidelines often recommend that in contract predicates of public member functions one should only use the public interface of the class. This is in case the class user needs to manually check if the contract is satisfied for an object whose state is not known. However, this is only a guideline, and enforcing it in the language would break other use cases that do not subscribe to the above advice.

In general, the users must *ensure* that the precondition of the called function is satisfied. If they do that, they do not have to check the precondition.

Allowing the access to protected and private members enables a practical usage scheme. In general, function precondition is something that cannot be fully expressed as C++ expression. The implementers choose how much of the function precondition they want to check. They may choose to check some parts of the precondition by accessing private members that they do not want to expose to the users, for instance, because the private implementation may change over time or under configuration:

```
class callback
{
    #if !defined NDEBUG
        mutable int _call_count = 0;
    #endif

    // ...

public:
    void operator() const
        // real contract: this function can be called no more than 8 times,
        // so the precondition is that the function has been called 7 or less times

    #if !defined NDEBUG
        // attempt to check the precondition
        PRE(_call_count <= 7);
    #endif
};
```

In the above example, the precondition can only be checked in debugging mode. Once `NDEBUG` is defined, member `_call_count` is removed and there is no way to test the precondition.

Also, a hypothetical constraint to use only public members in contract predicates could result in programmers turning their private and protected members into public members only to be able to express the pre- and postconditions, which does not sound like a good class design.

This has been described in detail in [\[P1289R1\]](#), and in fact adopted by EWG.

3.4. Naming the return value

{pro.ret}

It is possible to name the return value (or reference) in the postcondition, except for one situation: when we use a return placeholder type and do not provide the definition from which the type could be deduced:

```
auto get_digit()
    POST(c: is_digit(c)); // error: decltype(c) unknown
```

This has been discussed in detail in [\[P1323R2\]](#).

3.5. Indirect function calls

{pro.ptr}

Function pointers and references cannot have contract annotations, but functions with contract annotations can be assigned to them:

```
using fpa = int (*)(int) PRE(true); // error

using fptr = int (*)(int);
```

```

int (int i) PRE(i >= 0);

fptr fp = f;  // OK
fp(1);        // precondition is checked

```

In other words, contract annotations are not part of function type. Thanks to this decision, you can write code like:

```

int fast(int i) PRE(i > 0);
int slow(int i) PRE(true);  // no precondition

int f(int i)
{
    int (*fp) (int) = i > 0 ? fast : slow;
    return fp(i);  // if fast() is called, its precondition is checked
}

```

The consequence of this behavior is that an implementation cannot check the precondition in the place where function is called. The check has to be performed inside the function. Therefore, it is a reasonable implementation strategy to implement the precondition check inside the function in case it is called indirectly, but where the function is called directly, check it in the caller, in order to provide the source location that is the culprit of contract violation. This may sometimes result in checking the same precondition twice.

The same mechanism works for function wrappers:

```

using fp = int(*) (int);
int f(int i) PRE(i >= 0);

function<int(int)> fp = f;  // OK
fp(1);  // precondition is checked

```

3.6. Virtual functions

{pro.vir}

When a virtual function is overridden, the overriding function has the same set of preconditions and postconditions as the overridden function, whether the contract annotations are repeated in the overriding function or not:

```

struct Base {
    virtual void f() PRE(p1());
};

struct Deriv1 : Base {
    void f() override;           // ok: Deriv1::f has precondition p1()
};

struct Deriv2 : Base {
    void f() override PRE(p1()); // ok: Deriv2::f has the same precondition as Base
};

struct Deriv3 : Base {
    void f() override PRE(p2()); // error: Deriv3::f has different precondition than Base
};

```

You can omit contract annotations in the overriding function. The overriding function then has the same contract annotations as the virtual function in the base class, and the names in the predicates are looked up in the context of the base class.

```

static const int N = 1; // #1

struct Base
{
    virtual void f() PRE(N == 1);
};

template <int N>          // N is shadowed
struct Deriv : Base
{
    void f() override;
};

int main()
{
    Deriv<2>{}.f(); // precondition test passes
}

```

The precondition in the overriding function is `N == 1`, but the name lookup is performed in the context of class `Base`, so it sees the global variable `N` declared in line #1.

You cannot declare a contract annotation in the overriding function if the virtual function in the base class doesn't have a corresponding contract annotation.

You can declare a contract annotation in the overriding function, but it has to be identical (modulo the names of function parameters) to the corresponding contract in the overridden function. However, the program is ill formed, no diagnostic required, if name lookup in the predicate finds different entities than if the name lookup were performed in the context of the base class:

```
static const int N = 1; // #1

struct Base
{
    virtual void f()
        PRE(N == 1)
        POST(sizeof(*this) < 100);
};

template <int N>          // N is shadowed
struct Deriv : Base
{
    int i[100];

    void f() override
        PRE(N == 1)          // IF-NDR, finds different N
        POST(sizeof(*this) < 100); // IF-NDR, inspects different class
};
```

This means that we do not allow the preconditions in the overriding function to be "wider" and the postconditions to be "narrower" than in the overridden function, even though this idea — one aspect of the Liskov Substitution Principle — is well explored and implemented in other languages. The reason for this is that we do not yet have a good understanding of what effect this principle should have on the feature design. Should it be just a "best practice" that the programmers are taught? Or should it be enforced by the language? But how? We could think of a number of ways. Given the declarations:

```
struct Base {
    virtual void f() PRE(p1());
};

struct Deriv : Base {
    void f() override PRE(p2());
};
```

1. Should compiler be able to prove (even if it cannot see the definitions of `p1` and `p2`) that the latter is no stricter than the former?
2. Shall the compiler report an error unless the precondition in the overriding function has the form `p1() || p2()`?
3. Or should the compiler accept this code and evaluate `p1() && p2()` when `Deriv::f` is called through the `Base` interface, but evaluate predicate `p2()` when `Deriv::f` is called directly?

Option 1 is clearly impossible. The other options might be implementable, but it is more like a guess, as we know of no implementation experience with these.

However, the decision to add support for this feature can be deferred for later, because the way we specify the feature now (ill formed, no diagnostic required) remains open for future extensions in any of the three directions.

3.7. Contract annotations and the immediate context

{pro.imm}

In this proposal the predicates in contract annotations are *not* in the immediate context of the function. They behave similarly to exception specification:

```
template <std::regular T>
void f(T v, T u)
    PRE(v < u); // not part of std::regular

template <typename T>
constexpr bool has_f =
    std::regular<T> &&
    requires(T v, T u) { f(v, u); };

static_assert( has_f<std::string>);          // OK: has_f returns true
static_assert(!has_f<std::complex<float>>); // ill-formed: has_f causes hard instantiation error
```

As a consequence, we may have a function template that works well for a given type, but stops working the moment we add a contract annotation. This also affects how concepts would be taught: a good concept should express not only the operations that are necessary in the *implementation* of the generic algorithms, but also these that are necessary in the specification of contract annotations in these algorithms.

3.8. `abort()` vs `terminate()` {pro.end}

In this proposal, throwing from the predicate calls `std::terminate()` while a failed runtime check aborts the application even more abruptly: close to calling `std::abort()`, but we do not require the actual call to `std::abort()`, as the function may not be present in freestanding.

We do not encourage the implementations to allow the users to install custom contract violation handlers, nor do we specify any interface describing how this is done. However, we do not actively forbid the implementations from performing some logic, as long as it never throws or calls `longjmp()`.

The above distinction reflects the fundamental difference between the two situations. Throwing from the predicate is a random, unpredictable, but correct situation in the program. Maybe a comparison had to allocate memory, and this allocation failed, because today the server is exceptionally busy. We want to handle it the way we usually handle exceptions when there is no suitable handler: `std::terminate()` is an exception handler, with its unique control flow, however harsh.

In contrast, failing a runtime correctness test is an indication of a bug, and it is not clear if `std::terminate()`, which is the second level of exception handling mechanism, is a suitable tool. The call to `std::terminate()` either calls `std::abort()` or calls a terminate handler installed by the user. In case the contract is violated, and we can be sure the program contains a bug, calling a user-installed function may be unsafe, and can pose a security risk.

More, `std::terminate()` is not available in freestanding implementations.

3.9. No runtime error message {pro.log}

This revision of the paper does not require or encourage any error message to be displayed to standard diagnostic stream, or anywhere in *Eval_and_abort* mode. There are two reasons. First, there is no standard diagnostic stream on freestanding implementations, and we want contract support to be available on those platforms. Second, for security reasons. When an application is in a confirmed incorrect state, performing IO operations may pose a security risk. As the primary focus of this proposal is safety, we choose a conservative approach.

3.10. Contract annotations upon the first declaration {pro.dec}

We require that if a given function `f` has declared preconditions and postconditions, they shall be visible in the first declaration of `f` in a translation unit (TU): otherwise the program is ill-formed. Subsequent declarations can either omit contract annotations or repeat them in the identical form (modulo parameter names). If `f` is declared in more than one TU, the corresponding first declarations of `f` shall be identical (modulo parameter names): otherwise the program is ill-formed with no diagnostic required. As a consequence, the following is illegal:

```
int select(int i, int j); // first declaration

int select(int i, int j) // second declaration
    PRE(i >= 0)           // error: initial decl had different (no) contract annotations
    PRE(j >= 0)
    POST(r: r >= 0);
```

The reason for this restriction is implementability issues, similar to those for default function arguments.

4. Controversial aspects {con}

This section lists points of controversy inside SG21 for the recent contract design. For each of these points, we require a poll to be taken, to determine the group direction.

4.1. The choice of syntax {con.syn}

There are two visions for the syntax to describe contract annotations with significant support in SG21.

One is to use notation similar to attributes (but not 100% compatible with attributes):

```
int select(int i, int j)
    [[pre: i >= 0]]
    [[pre: j >= 0]]
    [[post r: r >= 0]] // r names the return value
{
    [[assert: _state >= 0]];

    if (_state == 0) return i;
```

```

    else                return j;
}

```

The other is to use notation similar to lambdas (but not 100% compatible with lambdas):

```

int select(int i, int j)
  pre{ i >= 0 }
  pre{ j >= 0 }
  post(r){ r >= 0 }      // r names the return value
{
  assert{ _state >= 0 };

  if (_state == 0) return i;
  else                return j;
}

```

The rationale for using the later syntax has been provided in [\[P2461R1\]](#). The analysis of pros and cons of using the former syntax has been provided in [\[P2487R0\]](#).

The primary argument in favor of quasi-attribute notation is to stress semantic characteristics similar to attributes. The common understanding of attributes is that they are hints for generating warnings or performing optimizations. Their removal, should not affect the correctness of the program (even though it is easy to construct an example using `no_unique_address` that contradicts this claim).

Contract annotations — at least one model thereof — shares similar features: they are hints for tools for generating warnings or emitting an instrumentation code. If these annotations are removed *from a correct program* (one that does not violate the declared contract annotations), this does not affect the correctness of the program.

The primary arguments in favor of quasi-lambda syntax is to avoid the problems reported for quasi-attribute syntax (e.g., that they look like attributes but do not follow other characteristics of attributes) and to offer an intuitive syntax for one of the future extensions: making copies of function arguments for use in postconditions.

4.2. Removal and duplication of side effects in predicates

{con.eff}

An implementation may need to evaluate the same predicate in a precondition twice. For direct calls, it is desired to insert the instrumentation code in the caller: this gives better diagnostics, so it is preferred whenever possible. However this is impossible when a function is called indirectly, either through a pointer or `std::function`: from the pointer signature we do not know if a function called has a precondition or not. Because of these cases the precondition check may need to be compiled into the function. This results in the situation where a function is called directly, but the body is in a different translation unit, and we want to make the check in the caller for better diagnostic, and inside the body to cover the indirect calls. We want to enable such implementation strategies. The end result for the programmer is that when the predicate has side effects, these effects occur twice.

For performance reasons an implementation may want not to evaluate the predicate, if it already knows what its result would be. This can happen when a function that produces a value has sufficiently similar postcondition `p` to the precondition of another function that subsequently consume the same value, and we do not see the body of `p`:

```

bool p(int); // defined in a different TU

int produce() POST(r: p(r));
void consume(int i) PRE(p(i));

int main() {
  consume(produce()); // can p() be called once
}

```

This seems redundant to call the same predicate twice. Of course, this seems so only if `p()` doesn't have side effects. If it does, and the program (or the programmer) relies on them, this elision can make the comprehension of the program harder, and the effects surprising.

Both [\[P2388R4\]](#) and [\[P2461R1\]](#) propose this capability to remove and duplicate the evaluation of the predicates. However, concerns have been expressed about it.

Note that [\[P2388R4\]](#) additionally proposes the "partial elision" of side effects in a predicate. We received a feedback that the group is against it, so this "partial elision" is not considered in this paper.

4.3. Function parameters in postconditions

{con.arg}

The model that we used for contract annotations is simple: we execute a predicate at certain specified points in time (for instance, just before the function call for preconditions) and the predicate observes the state of the program at that moment. This model works intuitively except for one case: when function:

1. has a non-reference parameter, and

2. this parameter is referenced in the postcondition, and
3. this parameter is modified inside the function.

This problem has been explained in detail in [\[P2388R4\]](#) and [\[P2466R0\]](#). Here, we only show an example that demonstrates the issue:

```
// declaration that the user sees:
int generate(int lo, int hi)
    PRE(lo <= hi)
    POST(r: lo <= r && r <= hi);

// definition that only the author sees:
int generate(int lo, int hi)
{
    int result = lo;
    while (++lo <= hi) // note: lo modified
    {
        if (further())
            ++result;    // incremented slower than lo
    }
    return result;    // at this point result < lo
}

// usage:
int min = 1;
int max = 10;

int r = generate(min, max); // postcondition check fails
assert(min <= r && r <= max); // even though this is satisfied
```

How is this problem addressed in other languages? In D, this problem has been ignored: postconditions like the one above give false positive or false negative results. In ADA this problem does not occur: this is due to the way the function arguments are designed. In ADA, for each function argument, the programmer has to specify if it is IN or OUT or INOUT. The OUT and INOUT parameters correspond to reference parameters in C++, so there is no problem here. The IN parameters, on the other hand, are immutable, so there is no question of changing them inside the function: IN parameters correspond to `const` by-value parameters in C++.

How can this issue be addressed?

1. [\[P0542R5\]](#) proposed to do what D does: it called this situation undefined behavior.
2. [\[P2388R4\]](#) and [\[P2461R1\]](#) proposes to imitate the solution from ADA: if the postcondition uses the non-reference parameters: require them to be `const` objects.

Additionally, [\[P2461R1\]](#) proposes for the future revisions the ability for the programmer to request that copies of designated function parameters be made. These copies could also be referenced when the postcondition is evaluated:

```
int generate(int lo, int hi)
    pre{ lo <= hi }
    post[lo, hi](r){ lo <= r && r <= hi };
```

The lambda-introducer syntax should make it immediately clear, even for the uninitiated programmers, that a copy of function parameters is being made.

The quasi-attribute notation also allows this as a future extension, however this would require a new notation, which comes with a complexity and an aesthetical cost:

```
int generate(int lo, int hi)
    [[pre: lo <= hi ]]
    [[post r, old_lo = lo, old_hi = hi: old_lo <= r && r <= old_hi ]];

// or some alternate notation

int generate(int lo, int hi)
    [[pre: lo <= hi ]]
    [[post r, =lo, =hi: lo <= r && r <= hi ]];
```

Thus, the decision how to address the issue of by-value arguments in postconditions is somewhat tied to the choice of syntax. But perhaps not as much as one might think at first.

We could allow postconditions to reference `const` by-value parameters in postconditions in the MVP. This would address a reasonable subset of real-life use cases immediately, and at the same time not prevent the addition of "copies on demand" in the future. We could go even further and require the compiler to do implicit copies for trivially-copyable types. This would create a "hybrid" solution that is the least intrusive for the programmers:

```

int generate(int lo, int hi)
  PRE(lo <= hi)
  POST(r: lo <= r && r <= hi) // ok: we can copy ints implicitly
{ /* ... */ }

BigInt generate(BigInt const lo, BigInt const hi)
  PRE(lo <= hi)
  POST(r: lo <= r && r <= hi) // ok: can safely inspect original objects
{ /* ... */ }

BigInt generate(BigInt lo, BigInt hi)
  PRE(lo <= hi)
  POST(r: lo <= r && r <= hi) // sorry: please, wait for C++29
{ /* ... */ }

```

Thus, the first question in this section is actually: how many cases of function arguments can we afford to handle in the MVP:

1. Disallow function arguments in postconditions: even the reference ones.
2. Disallow only by-value function arguments in postconditions: references are allowed.
3. Allow reference and const function arguments in postconditions.
4. Allow reference and const and trivially-copyable function arguments in postconditions.

Another question is a slight difference between [\[P2388R4\]](#) or [\[P2461R1\]](#). The former paper allows by-value parameter to omit `const` on the function declaration, but only requires it on function definition:

```

// P2388R4:

int generate(int lo, int hi)
  PRE(lo <= hi)
  POST(r: lo <= r && r <= hi); // ok: just a declaration

int generate(int const lo, int const hi) // ok: parameters are const in definition
{ /* ... */ }

```

The only thing we need here is that the compiler will ultimately prevent any modifications in the function body. In contrast, [\[P2461R1\]](#) requires that even the declarations have function arguments declared with `CONST`, even though programmers have been taught for years that the addition or the omission of `CONST` in by-value arguments in function declarations has no effect.

One reason for this additional restriction, is that the quasi-lambda solution may require certain mangling of function names in the presence of lambdas, and the lambda type that captures by reference may be affected based on whether by-value function arguments are `const` or not:

```

int generate(int lo, int hi)
  pre{ lo <= hi }
  post /*[&]*/ (r){ r: lo <= r && r <= hi }; // different mangling

int generate(int const lo, int const hi)
  pre{ lo <= hi }
  post /*[&]*/ (r){ r: lo <= r && r <= hi }; // different mangling

```

So the trade-off here is the ability to declare lambda-like copies of arguments at the cost introducing new rules for `CONST` by-value arguments in function declarations.

An additional question that one might want to answer is that if we allowed in postconditions arguments that are either references or `const` or trivially-copyable, are the remaining cases sufficient to motivate the addition of quasi-lambda syntax?

It should be noted [\[P2461R1\]](#) offers more motivation than the quasi-lambda syntax than solving the problem of function arguments in postconditions. The notable example is the support for "oldof" values:

```

void vector::push_back(T const& val)
  POST(r: size() == OLD_OF(size()) + 1);

// in P2461R1, post-MVP:

void vector::push_back(T const& val)
  post [old_size = size()]{ size() == oldof_size + 1 };

// in P2388R4, post-MVP:

void vector::push_back(T const& val)
  [[post old_size = size(): size() == oldof_size + 1]];

```


5. References

{ack}

This paper is a summary of SG21 discussions; all SG21 members contributed to this paper. John McFarlane suggested the idea to make implicit copies of trivially-copyable types.

6. References

{ref}

- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P1289R1] — J. Daniel Garcia, Ville Voutilainen, "Access control in contract conditions" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1289r1.pdf>).
- [P1323R2] — Hubert S.K. Tong, "Contract postconditions and return type deduction" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1323r2.html>).
- [P2388R4] — Andrzej Krzemiński, Gašper Ažman, "Minimum Contract Support: either No_eval or Eval_and_abort" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2388r4.html>).
- [P2461R1] — Gašper Ažman, Caleb Sunstrum, Bronek Kozicki, "Closure-Based Syntax for Contracts" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2461r1.pdf>).
- [P2466R0] — Andrzej Krzemiński, "The notes on contract annotations" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2466r0.html>).
- [P2487R0] — Andrzej Krzemiński, "Attribute-like syntax for contract annotations" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2487r0.html>).